

Ejemplos y ejercicios de OSTEP: Virtualización

Caps. 4 a 23 según el resumen `AllChapters.md` (el archivo termina en el cap. 23; el 24 es solo el diálogo/cierre). **resuelto** = ejemplo con desarrollo/resultado en el resumen. **tarea** = ejercicio propuesto por el libro (simulador/homework) que no viene resuelto. Cada uno indica con qué ejercicio del práctico se conecta.

Cómo usarlo: tapá el resultado, rehacé el ejemplo a mano y recién después abrí "ver resultado". Los ejemplos de los caps. 7 y 22 (marcados con ★) son los clásicos del libro; sus números los agregué de memoria y no están en el resumen, verificalos en el libro.

Parte I · CPU

Cap. 4 · Estados del proceso **resuelto**

Transiciones: Scheduled (Ready → Running), Descheduled (Running → Ready), I/O initiated (Running → Blocked), I/O completed (Blocked → Ready).

→ Práctico Ej. 10

Cap. 5 · fork/wait/exec, redirección **resuelto**

- fork: padre recibe PID del hijo, hijo recibe 0, error < 0; el orden padre/hijo no está determinado.
- Con `wait()` el hijo termina antes de que siga el padre.
- `wc archivo.c > salida.txt`: el hijo cierra stdout y abre el destino *antes* del exec (UNIX asigna el menor fd libre).
- `grep foo archivo | wc -l` con `pipe()`.

→ Práctico Ej. 4-9, 12

tarea `fork.py`: predecir el árbol de procesos para distintas secuencias de fork/exec; y programas que hagan fork+wait, fork+exec y redirección propia.

Cap. 6 · Limited Direct Execution **resuelto**

Flujo de una syscall: trap → nivel kernel, trap table (configurada al boot) → handler → return-from-trap. Con timer interrupt: el hardware guarda los regs de usuario en el kernel stack (guardado implícito) y `swtch` guarda los del kernel (explícito). Sin timer, un loop infinito monopoliza la CPU.

→ Práctico Ej. 11, 12(e)

tarea Medir el costo de una syscall y de un context switch (`lmbench -style`, con pipes entre dos procesos).

Cap. 7 ★ · Scheduling clásico **resuelto** (valores del libro)

Cap. 7 · Scheduling classes resuelto (valores del libro)

- 3 jobs de 10 s llegando juntos, FIFO: turnaround promedio = $(10+20+30)/3 = 20$.
- Convoy: A=100, B=C=10, FIFO → $(100+110+120)/3 = 110$.
- SJF sobre el mismo caso → $(10+20+120)/3 = 50$.
- B y C llegan en $t=10$ (A ya corriendo): SJF no apropiada (mal), **STCF** sí → turnaround promedio 50.
- Response time: 3 jobs de 5 s, SJF → $(0+5+10)/3 = 5$; RR con slice 1 → $(0+1+2)/3 = 1$ (pero turnaround peor).

Con I/O: cada ráfaga de CPU es un "sub-job"; mientras A hace I/O corre B (overlap).

→ Práctico Ej. 13, 14, 16, 18

tarea `scheduler.py` con FIFO/SJF/STCF/RR: calcular a mano turnaround y response y verificar con `-c`; `process-run.py` para el overlap CPU/IO.

Cap. 8 · MLFQ resuelto

Reglas finales 1–5 (prioridad, RR en el mismo nivel, job nuevo arriba, baja al agotar el allotment sin importar los yields, boost cada S). Problemas que motivan cada regla: starvation → boost; gaming (I/O justo antes del quantum) → mejor contabilidad; cambio de comportamiento → boost.

Configuración típica: quanta cortos arriba, largos abajo.

→ Práctico Ej. 17, 18(d)(e)

tarea `mlfq.py` : con 2–3 colas, trazar jobs de distinta duración, con y sin boost, con y sin gaming.

Cap. 9 · Proporcional resuelto

A=75 y B=25 tickets → 75 % / 25 % esperado (probabilístico en lottery). Stride: $\text{stride} \propto 1/\text{tickets}$, corre el menor pass value, exacto pero requiere estado global. CFS: menor vruntime, $\text{slice} = \text{sched_latency} / n$ (mínimo `min_granularity`), `nice -20...19` cambia el peso.

El práctico no lo toma, pero podría aparecer como V/F.

tarea `lottery.py` y `stride.py` : ¿cómo mejora la fairness al crecer el tiempo de corrida?

Parte II · Memoria**Cap. 13–14 · Address space y API** resuelto

Layout: código arriba de todo, heap crece hacia abajo, stack hacia arriba. `int *x = malloc(sizeof(int))` son dos reservas: el puntero en el stack, el int en el heap. Errores típicos: olvidar reservar, poco espacio (sin `+1` para `\0`), leak, free doble, free de puntero inválido, usar después de liberar. Herramientas: gdb, valgrind.

Práctico Ej. 1, 2, 3

→ Práctico Ej. 1, 2, 3

tarea Programas con bugs para depurar con valgrind (`null.c` , `leak` , `free` parcial).

Cap. 15 · Base y bounds **resuelto**

Espacio de 4 KB cargado en 16 KB: virtual 0 → 16 KB; 1 KB → 17 KB; **3000** → **19384**; **4400** → **fault**. El código `128: movl..., 132: addl..., 135: movl...` muestra que cada fetch, load y store se traduce.

→ Práctico Ej. 4-7

tarea `relocation.py` : dadas base/bounds y direcciones, decidir cuáles son válidas.

Cap. 16 · Segmentación **resuelto**

Tabla: Code 32K/2K, Heap 34K/3K, Stack 28K/2K. Virtual 100 (código) → 32868. Virtual 4200 (heap): offset 4200-4096 = 104 → 34K+104 = **34920**. Stack crece negativo: virtual 15 KB → offset crudo 3 KB - 4 KB = -1 KB → 28K-1K = **27 KB**. Selección de segmento con los bits altos: `Seg = (VA & SEG_MASK) >> SEG_SHIFT` , `Offset = VA & OFFSET_MASK` , si `Offset ≥ Bounds` → fault.

→ Práctico Ej. 8

tarea `segmentation.py` .

Cap. 17 · Espacio libre **resuelto**

Heap de 30 bytes: 10 libres, 10 usados, 10 libres → 2 nodos en la lista (splitting/coalescing). Heap de 4096 de `mmap` : un nodo con `size = 4096 - sizeof(node_t)` . El header guarda `size+magic` y `free` usa `(header_t*)ptr - 1` ; pedir N reserva N + header. Libres 10, 30, 20 y pedido de 15: **best** → 20 (resto 5); **worst** → 30 (resto 15); **first** → 30.

→ Práctico Ej. 9 (mismo método, otros números)

tarea `malloc.py` : correr best/worst/first con listas ordenadas por tamaño/dirección y con coalescing.

Cap. 18 · Paginación lineal **resuelto**

Espacio de 64 B, páginas de 16 B, 8 frames. Tabla VP0 → PF3, VP1 → PF7, VP2 → PF5, VP3 → PF2. Virtual 21 = 010101 → VPN 01, offset 0101; VP1 → PF7 (111) → 1110101 = **117**. Tabla lineal de 32 bits con 4 KB: $2^{20} \text{ PTE} \times 4 \text{ B} = \mathbf{4 \text{ MB por proceso}}$. El pseudocódigo de traducción y el trace de un loop muestran las referencias extra a la tabla.

→ Práctico Ej. 12

tarea `paging-linear-translate.py` : traducir direcciones con distintos tamaños de espacio/página.

Cap. 19 · TLB **resuelto**

Arreglo de 10 ints (4 B), páginas de 16 B: patrón `miss, hit, hit, miss, hit, hit, hit, miss, hit, hit` = **70 %** de hit rate (localidad espacial). Repetir el recorrido mejora por localidad

temporal. LRU en loop de $n+1$ páginas con TLB de $n \rightarrow 0\%$ de hits. Flush vs ASID en el context switch. MIPS R4000: VPN 19 bits, ASID 8 bits, TLB por software.

→ Práctico Ej. 10, 11

tarea `tlb.c` : medir el costo de un acceso variando el número de páginas tocadas y detectar el tamaño del TLB.

Cap. 20 · Tablas chicas **resuelto**

Páginas de 16 KB: 18 bits de VPN $\rightarrow$ 1 MB por tabla (4× menos, pero fragmentación interna).

Multinivel: espacio de 16 KB, páginas de 64 B $\rightarrow$ VPN de 8 bits, tabla lineal de 1 KB = 16 páginas de tabla de 16 PTE, directorio de 16 entradas (4 bits). Solo 3 páginas (1 directorio + 2 tablas) en lugar de 16. Traducción de **0x3F80**: PD index 1111 (15) $\rightarrow$ PT index 1110 (14) $\rightarrow$ PFN 55 $\rightarrow$ $(55 \ll 6) + 0 = 0x0DC0$. **Tres niveles:** espacio de 30 bits, páginas de 512 B $\rightarrow$ 128 PTEs por página $\rightarrow$ 7 bits por nivel: (7,7,7,9). Costo: 2 accesos extra en un TLB miss.

→ Práctico Ej. 13, 14, 15

tarea `paging-multilevel-translate.py` y comparar tamaño de tabla lineal vs multinivel.

Cap. 21 · Swapping **resuelto**

Present bit; page fault = acceso legal a una página no presente. Handler: buscar frame libre $\rightarrow$ si no hay, `EvictPage()` $\rightarrow$ `DiskRead` (proceso blocked) $\rightarrow$ `present=1`, PFN $\rightarrow$ reintentar. Tres casos en un TLB miss: válida y presente / válida no presente / inválida. Watermarks HW/LW con un daemon en background y clustering de escrituras. Fig. 21.1: 4 páginas físicas y 8 bloques de swap.

→ Práctico Ej. 19(b)

tarea `vmstat` y `mem.c` : observar swap in/out al pedir más memoria que la RAM.

Cap. 22 ★ · Políticas de reemplazo **resuelto**

$AMAT = T_M + P_{miss} \cdot T_D$. Con $T_M=100$ ns, $T_D=10$ ms: $P_{miss}=0,1 \rightarrow \approx 1$ ms; $P_{miss}=0,001 \rightarrow \approx 10,1$ μ s.

Traza 0,1,2,0,1,3,0,3,1,2,1 con 3 frames: **OPT** 6 hits/5 misses = 54,5 % (los 3 primeros son compulsory); **FIFO** 36,4 %; **LRU** iguala a OPT.

Belady con FIFO: 1,2,3,4,1,2,5,1,2,3,4,5 empeora al pasar de 3 a 4 frames (LRU tiene stack property).

Workloads: sin localidad (todas iguales), 80-20 (LRU mejor), looping secuencial de 50 páginas (LRU y FIFO 0 % incluso con 49 frames; Random mejor).

Reloj: use bit, la aguja limpia bits en 1 hasta encontrar uno en 0; se prefiere víctima clean antes que dirty.

→ Práctico Ej. 19(b); muy probable en V/F

► **Hacelo vos: FIFO en la secuencia de Belady**

tarea `paging-policy.py` : traza con FIFO, LRU, OPT, Random, CLOCK y variando el tamaño de cache.

Cap. 23 · VAX/VMS y Linux resuelto

VAX: páginas de 512 B → VPN de 23 bits; dos bits altos eligen segmento (P0, P1, S). Página 0 inaccesible (null pointer), kernel mapeado en cada address space. Sin reference bit → se emula con permisos y un trap. **Segmented FIFO** (RSS por proceso) + listas de segunda oportunidad (clean/dirty) + clustering. Lazy: **demand zeroing** y **COW** (read-only en ambos; al escribir, trap y copia). Linux: corte user/kernel en 0xC0000000 (32 bits), kernel logical (`kmalloc` , mapeo directo, no swappable, apto DMA) vs virtual (`vmalloc`); 4 niveles con 36 bits de traducción; huge pages 2 MB/1 GB; page cache con 2Q (inactive/active); mmap; NX, ROP, ASLR/KASLR; Meltdown/Spectre y KPTI.

→ Práctico Ej. 19(a)(c)(d)(e)(f)(g)(i)(j)

Cierre

Si tenés que priorizar: **(1)** cap. 7–8 con líneas de tiempo a mano, **(2)** caps. 15–16 y 20 con traducciones numéricas, **(3)** cap. 19 y 22 (hit rate, AMAT, traza FIFO/LRU/OPT/reloj). Coinciden casi 1 a 1 con los ejercicios del práctico.